TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 5 – Software Test Documentation

Topic: Customers Management System for NAWACO

Project Code: CMSN

Document Code: CMSN- v1.4

– Hanoi, Jan 2026 –

Table of Contents

I. Record of Changes..... 3
II. Testing Documentation..... 4
1. Scope of Testing..... 4
1.1 Stage of testing..... 4
1.2 Type of Testing..... 4
1.3 Range of testing..... 6
2. Test Strategy..... 6
2.1 Testing model..... 6
2.2 Testing Types and Levels..... 10
2.3 Supporting Tools..... 11
3. Test Plan..... 13
3.1 Human Resources..... 13
3.2 Test Environment..... 14
3.3 Test Milestones..... 14
4. Test Cases..... 15
4.1 Unit test..... 15
4.2 Integration test..... 15
4.3 System test..... 15
4.4 Non-functional test..... 15
4.5 User Acceptance test..... 15
5. Test Reports..... 15
5.1 Unit test..... 15
5.2 Integration test..... 17
5.3 System test..... 17
5.4 User Acceptance Test..... 18

I. Record of Changes

Date	A* M, D	In charge	Change Description
10/04/2026	A	GiangPT	Add Testing document
12/04/2026	M	GiangPT	Modified Type of testing
12/04/2026	A, M	GiangPT	Modified testing model.add V-Model structure
20/04/2026	M	GiangPT	Modified testing document

*A - Added M - Modified D - Deleted

II. Testing Documentation

1. Scope of Testing

1.1 Stage of testing

There are 3 level of testing utilized in our CMSN project, they are Unit testing, Integration testing and System testing

ID	Test Stages	Description
1	Unit testing	This stage targets the smallest testable parts of the application, such as methods or classes, and verifies their correctness in isolation. It typically employs white-box testing techniques to ensure the internal logic of each component performs accurately as intended by the developer.
2	Integration testing	Integration testing combines multiple units and tests them as a group. The goal is to identify defects that arise when units or modules interact with each other, ensuring that the integrated components work well together. This testing method helps detect issues with interfaces and interactions between modules
3	System testing	System testing is the first phase where the entire system is tested as a whole. It aims to validate that the system meets all the outlined requirements and complies with Quality Standards. This type of testing focuses on the overall system functionality rather than individual components.
4	User Acceptance Test	UAT (User Acceptance Testing) is the phase of software testing performed directly by end users or customers. The purpose of UAT is to verify that the system meets the specified requirements and can be used effectively in real-world scenarios. If the software passes UAT, it means the product is ready for official deployment.

Table 1: Stages of testing

1.2 Type of Testing

Type of Tests	Test Levels			
	Unit	Integration	System	User Acceptance
Functional Testing	X	X		
Non-functional Testing		X	X	X

Table 2: Types of testing

1.2.1 Functional testing

Functional testing verifies that individual functions or features of the system operate according to defined requirements. Each test case is designed to validate various logical paths, including different input conditions and boundary scenarios.

- All possible branches of logic within a function or method should be covered using diverse input data.
- The testing process is considered successful if all documented test cases execute and pass without failure.

1.2.2 Non-Functional testing

This testing focuses on the visual and usability aspects of the application to ensure a positive user experience and interface compliance with design specifications.

Performance Testing

- Assesses how the system behaves under expected and peak load conditions. This includes:
- **Response Time**: How quickly the application responds to user actions or API requests.
- **Throughput**: Number of transactions processed within a given time frame.

Scalability Testing

- Determines the system's ability to handle increased workloads, such as more users or larger data volumes, without degrading performance.

Reliability Testing

- Ensures the system continues to operate correctly over time and under stress. It includes:
- **Stability Over Time**: Application can run for extended periods without crashing.
- **Error Recovery**: The system recovers gracefully from failures or interruptions.

Security Testing

- Assesses the system’s ability to protect data and resist malicious actions. This includes:
- **Authentication & Authorization**: Ensuring only legitimate users can access resources.
- **Data Protection**: Validating encryption, secure transmission, and safe data storage.
- **Vulnerability Assessment**: Detecting and addressing potential security flaws.

1.3 Range of testing

Our team aims to cover all the functionality described in the SRS and optional features that may be added in the future.

2. Test Strategy

2.1 Testing Types

2.1.1 Testing model overview

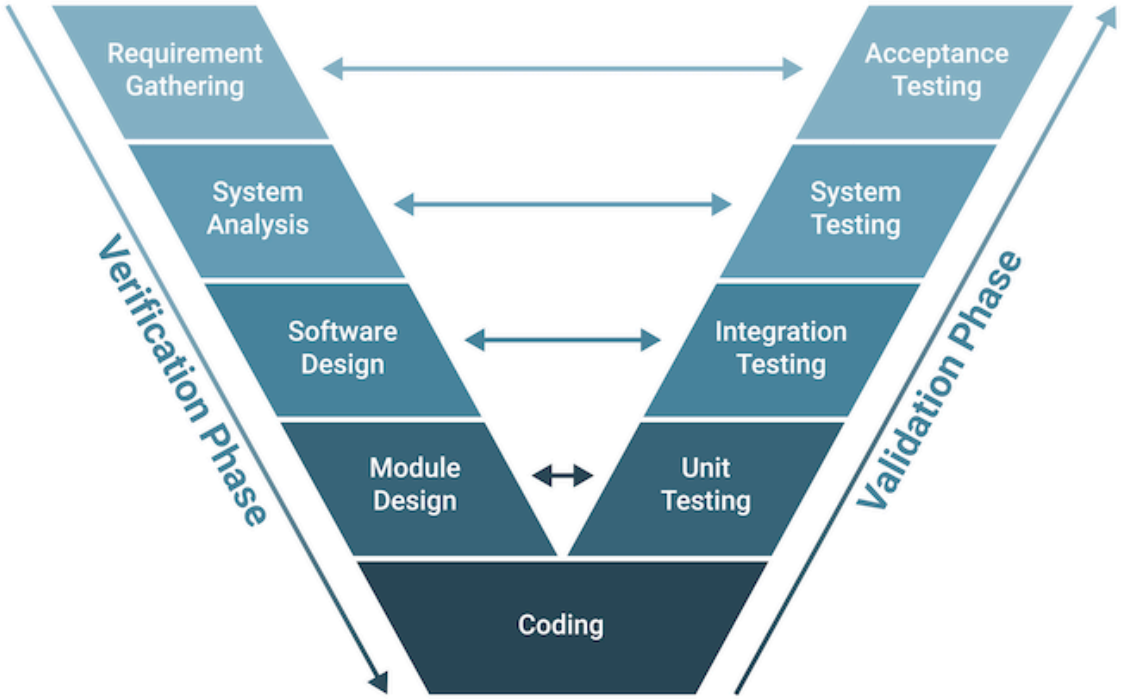


Figure 1: V-Model

The CMSN project adopts the V-Model (Verification and Validation Model) for software development and testing. The V-Model is a structured development methodology that emphasizes a strict relationship between development phases and corresponding testing phases. For every development activity, there is a corresponding testing activity, ensuring that verification and validation are conducted in parallel throughout the lifecycle.

This model enhances traceability, improves quality assurance, and ensures that issues are detected at the earliest possible stage by validating each deliverable against its defined requirements.

2.1.2 V-model structure in CMSN

Verification Phases (Left Side of V):

- **Requirements Analysis:**
  Define and document user requirements. These are validated through acceptance testing later in the cycle.

- **System Design:**
  Specify system architecture and high-level modules. This phase maps to system testing.
- **High-Level Design:**
  Design the structure of sub-systems and their interfaces. It corresponds to integration testing.
- **Low-Level Design:**
  Define detailed design for each module or component. This is linked to unit testing.
- **Implementation (Coding):**
  Actual development of components begins after all design phases are complete.

Validation Phases (Right Side of V):

- **Unit Testing:**
  Conducted to validate individual components against their detailed design specifications.
- **Integration Testing:**
  Ensures that module interactions function correctly according to the high-level design.
- **System Testing:**
  Verifies the complete system behavior against system design and requirements.
- **Acceptance Testing:**
  Validates the software against user requirements, ensuring the system meets business goals.

2.1.3 Key Characteristics

- **Clear Mapping Between Phases:**
  Every development phase has a corresponding testing phase, ensuring alignment between design and validation.
- **Early Test Planning:**
  Testing activities are planned during the early stages of development, improving preparation and efficiency.
- **Strict Process Control:**
  The model follows a sequential flow, minimizing ambiguity and enhancing project tracking.
- **High Traceability:**
  Each requirement is traceable through the entire development and testing process.
- **Defect Prevention and Early Detection:**
  Validation begins from the start, reducing the risk of propagating defects.

2.1.4 Advantages of V-Model

- **Improved Quality Assurance:**
  By validating each development phase through corresponding testing activities, quality is assured at every level.
- **Early Identification of Defects:**
  As testing is planned early, potential issues are identified during the design stages.
- **Better Documentation and Control:**
  Each phase has well-defined deliverables, making it easier to manage and audit.
- **Enhanced Stakeholder Confidence:**
  Clearly structured processes and early visibility into testing provide transparency for stakeholders.
- **Minimized Rework:**
  Early validation reduces the cost and effort of fixing errors discovered late in the process.

2.1.5 How the model is applied

- **Requirement Analysis and Acceptance Test Design:**
  User requirements are defined and simultaneously used to plan acceptance tests.
- **System Design and System Testing Preparation:**
  Architecture and overall system behavior are specified, and system test plans are created accordingly.
- **Detailed Design and Integration Test Design:**
  Each module's interactions are designed and mapped to integration test cases.
- **Coding and Unit Testing:**
  Individual modules are developed and tested against the low-level design to ensure correct functionality.
- **Sequential Testing Execution:**
  After coding is complete, tests are executed in order: unit => integration => system => acceptance.
- **Final Delivery:**
  Upon passing all levels of testing, the complete system is deployed after formal validation.

2.1.6 Integrating V-Model Testing within the Iterative & Incremental Development Approach

While the CMSN project adopts the Iterative and Incremental Model for software development, it integrates the V-Model as the foundation for test planning and execution within each iteration. This hybrid approach leverages the strengths of both models: the flexibility and adaptability of iterative development, and the structured, phase-aligned testing discipline of the V-Model.

How Integration Works

- **Per-Increment V-Model Application:**
  Each increment in the iterative cycle follows a mini V-Model. Requirements defined for the increment are immediately paired with corresponding test activities:
    - Requirement Analysis => Acceptance Test Planning
    - Design Specification => System & Integration Test Design
    - Module Design => Unit Test Preparation
    - Coding => Unit Test Execution
- **Iteration-Based Testing Discipline:**
  At the end of each iteration, testing is carried out in the order defined by the V-Model:
    - Unit testing for individual components
    - Integration testing for new and existing modules
    - System testing for the full set of implemented features
    - Acceptance testing where applicable

- **Feedback-Driven Updates:**
  Defects found in later testing stages (system or acceptance) inform refinement of earlier phases in the next iteration, maintaining the iterative feedback loop while preserving V-Model rigor.

Regression Testing Integration:

After each increment, regression testing is performed to ensure previously validated features are not broken by new changes — aligning with the V-Model's emphasis on validation and verification.

Benefits of the Combined Approach

- **Balanced Structure and Flexibility:**
  The V-Model ensures quality through phase-based validation, while iterative cycles allow for continuous improvement and adaptation to evolving requirements.
- **Improved Test Planning in Each Iteration:**
  V-Model discipline allows test cases and test data to be well-prepared and traceable even in small, frequent releases.
- **Quality Assurance Across Increments:**
  Testing is not just reactive but proactively planned from the start of each cycle, leading to better test coverage and early defect discovery.
- **Scalable and Manageable Testing Process:**
  By using the V-Model within iterations, testing remains organized and scalable, even as the project grows in complexity.

2.1.7 Conclusion

This hybrid approach enables the CMSN project to benefit from the agility and feedback loop of the Iterative & Incremental model, while maintaining quality assurance through the

structured testing framework of the V-Model. Each iteration becomes a self-contained development and testing cycle with clear traceability, test accountability, and measurable progress — making it a highly effective strategy for delivering robust and reliable systems in dynamic project environments.

2.2 Testing Types and Levels

The testing activities in the project follow a hybrid approach, combining the V-Model with the Iterative & Incremental Development Model. This approach ensures that:

- Verification and validation activities occur in parallel with development.
- Testing is planned and conducted incrementally, aligning with each iteration.
- Each increment undergoes a full testing cycle, enabling early defect detection and continuous quality improvement.

2.2.1 Unit Testing

Purpose: To validate individual units such as functions, methods, or classes.

- Executed by developers during each iteration.
- Verifies that components work in isolation.
- Uses test data to cover multiple logic branches (branch coverage).
- Includes error-handling scenarios (e.g., invalid input or constraint violations).

Testing coverage: The testing cover rate must be at least 90%. (Ex: the Unit has 100 lines of code so the test must execute all 100 lines of code). If fail the unit will be considered as not working well with business requirements and must be fixed to execute the next level of testing process.

2.2.2 Integration Testing

Purpose: To test interactions between integrated components or services.

- Performed after unit testing within the same iteration.
- Ensures that combined modules interact correctly.
- Common in API-level testing and database interaction.

Testing coverage: The testing cover rate must be at least 100% (Ex: the project have 10 modules so the testing process must test the connection of all 10 modules). When this level done, the next level could be executed, if fail the process must be stopped to

2.2.3 System Testing

Purpose: To validate the complete and integrated system.

- Conducted once a functional increment is fully developed.
- Verifies that the system meets functional and non-functional requirements.
- Includes end-to-end test scenarios simulating real-world usage.

Testing coverage: The testing must reach 100% if all features exist in the project or else the project could not be published.

2.2.4 User Acceptance Testing

Purpose: To confirm the system fulfills business requirements and user expectations.

- After system testing, by end users or business representatives.
- That the system supports real-world business workflows and delivers expected outcomes.
- Executing actual business scenarios to ensure readiness for production.

Testing coverage: All critical business processes must be tested and accepted by users; otherwise, the system cannot go live.

2.3 Supporting Tools

2.3.1 Microsoft Excel

Figure 2: Excel

- **Description**: Microsoft Excel is a spreadsheet program from Microsoft, widely used for data management and analysis, and part of the Office product suite.
- **Purpose**:
  - Manage and organize test cases systematically.
  - Track test execution status and results.
  - Document test data and prepare summary reports for testing phases.

2.3.2 Postman

Figure 3: Postman

- **Description**: Postman is an API platform that simplifies building, testing, and managing APIs
- **Purpose**:
  - Manage and document the list of all APIs in the project.
  - Perform manual testing of API endpoints.
  - Validate API responses against expected results and analyze request-response workflows.

2.3.3 Chrome DevTools

Figure 4: Chrome DevTools

- **Description**: Chrome DevTools is a set of tools for web developers, built into the Google Chrome browser.
- **Purpose**:
  - View logs and debug web applications directly.
  - Inspect and edit HTML, CSS right in the browser.
  - Analyze requests, test performance and debug JavaScript.

3. Test Plan

3.1 Human Resources

Worker/Doer	Role	Specific Responsibilities/Comments
GiangPT	Project Manager	- Develops and maintains project schedules to ensure timely completion of tasks, including testing phases. - Reviews test cases and ensures alignment with project requirements and business goals. - Examines test reports to monitor progress, identify risks, and maintain quality. - Communicates testing outcomes and project updates to stakeholders. - Manages risks during testing and collaborates with the team to mitigate them.
GiangPT DungND	Tester	- Executes system testing to ensure the system meets functional and non-functional requirements. - Manages test environments, tools, and resources, and assigns test tasks if applicable. - Develops a comprehensive Test Plan that defines objectives, strategies, schedules, and deliverables. - Creates detailed Test Cases covering functionality, business rules, and edge cases. - Logs defects, tracks progress, and prepares Test Reports summarizing results. - Ensures compliance with testing standards and validates the system against user requirements.
DungND HieuDM HoanTX DangPH	Developers	- Develops and executes unit tests to validate individual components or modules. - Writes integration test scripts to ensure proper interactions between system components. - Resolves bugs reported during testing and collaborates with testers for retesting. - Refactors and optimizes code to enhance performance, maintainability, and compliance with standards. - Documents test scripts, resolved issues, and changes for knowledge sharing and transparency.

Table 3: Human resources

3.2 Test Environment

Purpose	Tool	Provider	Version
Database	PostgreSQL		V9.8
Browser	Chrome/Edge	Google	139.0.7258.127/139.0.3405.86
Host	https://capstone-project-chi-rouge.vercel.app/login		
Testing Framework	JUnit	JUnit.org	v5.13.4
Static Code Analysis	IntelliJ IDEA	JetBrains	2024.01.2

Table 4: Table Test Environment

3.3 Test Milestones

Milestone Task	Start Date	End Date
Create test plan	05/01/2026	06/01/2026
Review and update test plan	05/04/2026	20/04/2026
Iteration 1	26/01/2026	15/02/2026
Create unit test cases	26/01/2026	01/02/2026
Execute unit test	26/01/2026	01/02/2026
Create integration test and system test cases	26/01/2026	01/02/2026
Execute integration test and system test	26/01/2026	01/02/2026
Review and update test plan	05/02/2026	15/02/2026
Iteration 2	16/02/2026	15/03/2026
Create unit test cases	16/02/2026	01/03/2026
Execute unit test	17/02/2026	01/03/2026
Create integration test and system test cases	20/02/2026	01/03/2026
Execute integration test and system test	21/02/2026	01/03/2026
Review and update test plan	08/03/2026	15/03/2026

Iteration 3	16/03/2026	05/04/2026
Create unit test cases	16/03/2026	28/03/2026
Execute unit test	18/03/2026	28/03/2026
Create integration test and system test cases	21/03/2026	28/03/2026
Execute integration test and system test	23/03/2026	28/03/2026
Overall Review	29/03/2026	05/04/2026

Table 5: Table Test Milestones

4. Test Cases

4.1 Unit test

Reference: Report5.1_UnitTest_v1.0

4.2 Integration test

Reference: Report5.2_IntegrationTest_v1.0

4.3 System test

Reference: Report5.3_SystemTest_v1.0

4.4 Non-functional test

Reference: Report5.4_NonFunctionalTest_v1.0

4.5 User Acceptance test

Reference: Report5.5_UserAcceptanceTest_v1.0

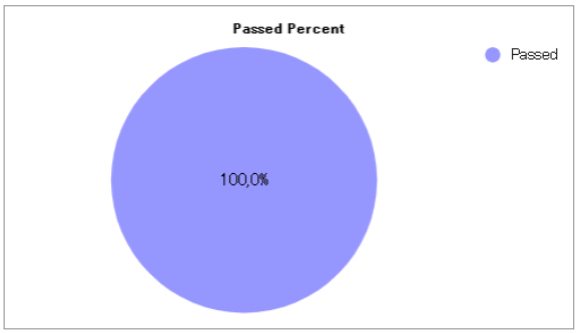
5. Test Reports

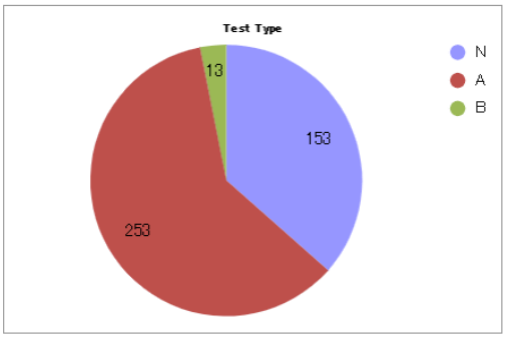
5.1 Unit test

This section outlines the structure of unit tests for each service in the system. The test classes shown above cover core functionalities using isolated test cases. For each service, test reports including test case count, result status, and sample evidence will be provided to demonstrate the effectiveness and correctness of the implemented logic.

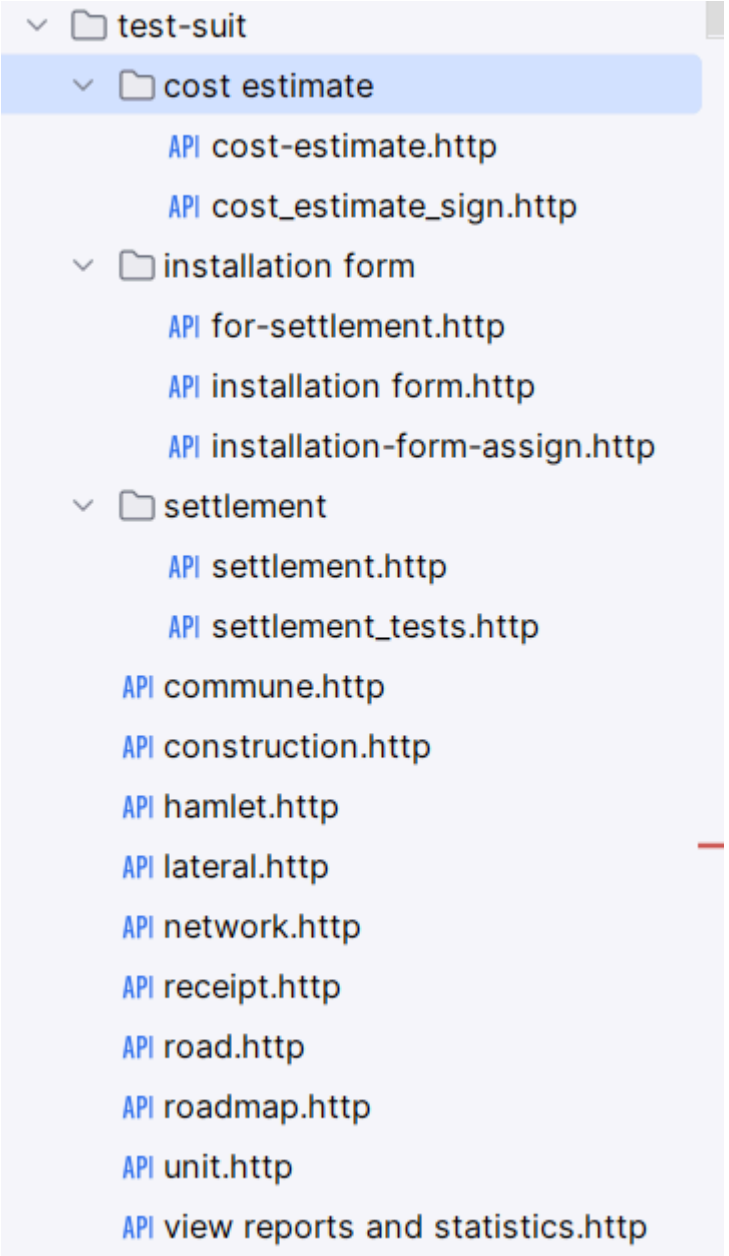
Sub total	419	0	0	153	253	13	419

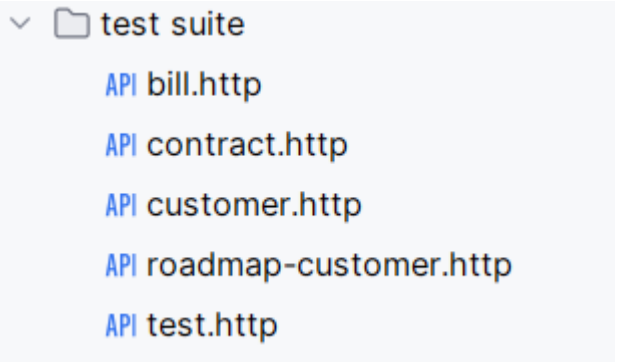
Test coverage	**100,00 %**
Test successful coverage	**100,00 %**
Normal case	**36,52 %**
Abnormal case	**60,38 %**
Boundary case	**3,10 %**

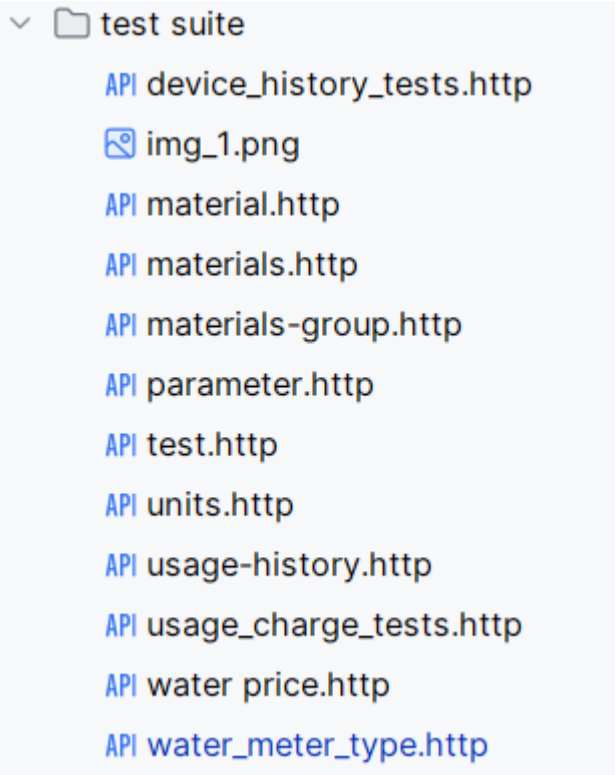


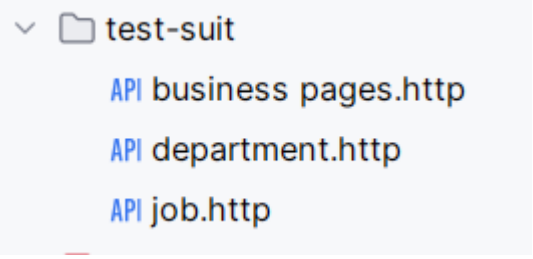


5.1.1.Report of unit testing


test-suit
cost estimate
cost-estimate.http
cost_estimate_sign.http
installation form
for-settlement.http
installation form.http
installation-form-assign.http
settlement
settlement.http
settlement_tests.http
commune.http
construction.http
hamlet.http
lateral.http
network.http
receipt.http
road.http
roadmap.http
unit.http
view reports and statistics.http


test suite
bill.http
contract.http
customer.http
roadmap-customer.http
test.http





5.2 Integration test

This section presents the results of integration testing, which verifies the interaction between different system modules. The reports below include both white-box testing (via direct API calls using Postman) and black-box testing (based on functional flows from the user's perspective). Each row in the report reflects a specific integration scenario, with metrics showing the number of test cases executed, passed, or pending. Full coverage and 99,29% pass rate demonstrate that the integrated components work together as intended.

TEST STATISTICS

Project Name	*Customers Management System for NAWACO*			GiangPT
Project Code	*CMSN*			
Document Code	*CMSN_IntegrationTest_v1.0*	**Issue Date**		*10/01/2026*
Notes	*<List modules included in this release> ex: Release 1 includes 2 modules: Module1 and Module2*			

No	Module code	Passed	Failed	Pending	N/A	Number of test cases
1	Auth Management	47	0	0	0	47
2	Roadmap Management	27	0	0	0	27
3	Lateral Management	22	0	0	0	22
4	Commune Management	29	0	0	0	29
5	Neighborhood Unit Management	27	0	0	0	27
6	Road Management	25	0	0	0	25
7	Hamlet Management	28	0	0	0	28
8	Water Meter Types Management	53	0	0	0	53
9	Materials Group Management	32	0	0	0	32
10	Materials Price Management	42	0	0	0	42
11	Unit Management	30	0	0	0	30
12	System Parameters Management	14	0	0	0	14
13	Water Price Management	43	0	0	0	43
14	Job Management	33	0	0	0	33
15	Department Management	38	0	0	0	38
16	Water Supply Network Management	33	0	0	0	33
17	New Installation Requests Management	99	0	0	0	99
18	Receipt Management	40	0	0	0	40
19	Customers Management	35	0	0	0	35
20	Employees Management	32	0	0	0	32
21	Cost Estimations Management	30	0	0	0	30
22	Contract management	23	0	0	0	23
23	Construction requests Management	22	0	0	0	22
24	Settlements Management	36	0	0	0	36
	Sub total	840	0	0	0	840

Test coverage **100,00 %**
Test successful coverage **100,00 %**

5.2.1:Integration Testing Result

5.3 System test

This section presents the results of integration testing, which verifies the interaction between different system modules. The reports below include both white-box testing (via direct API calls using Postman) and black-box testing (based on functional flows from the user's perspective). Each row in the report reflects a specific integration scenario, with metrics showing the number of test cases executed, passed, or pending. Full coverage and 100% pass rate demonstrate that the integrated components work together as intended.

No	Module code	Passed	Failed	Pending	N/A	Number of test cases
1	Installation	19	0	0	0	19
2	Meter Inspection	5	0	0	0	5
	Sub total	24	0	0	0	24

Test coverage **100,00 %**
Test successful coverage **100,00 %**

5.3.1: System Testing Result

5.4 User Acceptance Test

This section presents the results of user acceptance testing (UAT), which validates the system against business requirements and usability standards. Each test case corresponds to a specific requirement ID, and covers critical features across project management, schedule views, guest access, and user interface behaviors. All acceptance criteria were met with 100% test coverage, ensuring the product is fully aligned with user expectations and is ready for client sign-off.

No.	Module	ID	Checklist	Yes	No
1	Authorization and Authentication	CL-001	The system allows users to log in and log out safely.	X	
2		CL-002	Password reset via email link works correctly.	X	
3		CL-003	Password complexity requirements enforced.	X	
4		CL-004	Passwords must be strong (include letters, numbers, and special characters).	X	
5	Materials and Equipment Management	CL-005	he system displays lists of materials, equipment, and related entities (roadmaps, laterals, communes, units, etc.) correctly.	X	
6		CL-006	Users can create new records (roadmap, lateral, commune, neighborhood unit, road, hamlet, material price, water meter type, material group, unit, system parameter, water price) with valid data.	X	
7		CL-007	Users can update existing records with valid data and changes are saved correctly.	X	
8		CL-008	Users can delete records (roadmap, commune, neighborhood unit, etc.) and the system updates the list accordingly.		
9		CL-009	System validates input data (required fields, format, duplicate values) when creating/updating records.	X	
10	Internal Company Human Resources Management	CL-010	The system displays lists of departments, branches, employees, and jobs correctly.	X	
11		CL-011	Users can create and update departments, branches, employees, and job information.	X	
12		CL-012	Users can delete departments, branches, employees, and jobs where applicable.	X	
13		CL-013	System enforces validation rules and permission control when managing HR data.	X	
14	Customer's Water Meter Lifecycle Management	CL-014	The system displays installation requests, receipts, cost estimates, contracts, settlements, and related lists correctly.	X	
15		CL-015	Users can create, update, approve/reject, and assign requests through the workflow.	X	
16		CL-016	Users can delete or cancel records (requests, estimates, contracts, settlements) where permitted.	X	
17		CL-017	Workflow transitions (approve, reject, assign, accept construction, sign confirmation) are processed correctly and reflected in system status.	X	
18	Meter Recording & Billing Workflow Management	CL-018	The system allows users to view and validate meter readings, ensuring data accuracy and applying validation rules.	X	
19		CL-019	The system displays customer lists by roadmap, customer details in recording periods, payment information, and bills correctly.	X	
20		CL-020	Users can update meter readings (including mobile input), and the system processes billing data accurately; users can also delete or adjust incorrect readings/bills where permitted.	X	

Test coverage	**100.00 %**
Test successful coverage	**100.00 %**

5.4.1: Acceptance Testing Result